

Unidad 5. Desarrollo de componentes de un ERP-CRM

1. Introducción

Para poder desarrollar adecuadamente un módulo es recomendable el empleo de un entorno adecuado de desarrollo como Visual Studio Code, Atom o PyCharm ya que se trabajará con ficheros .py (Python), .xml y .csv directamente.

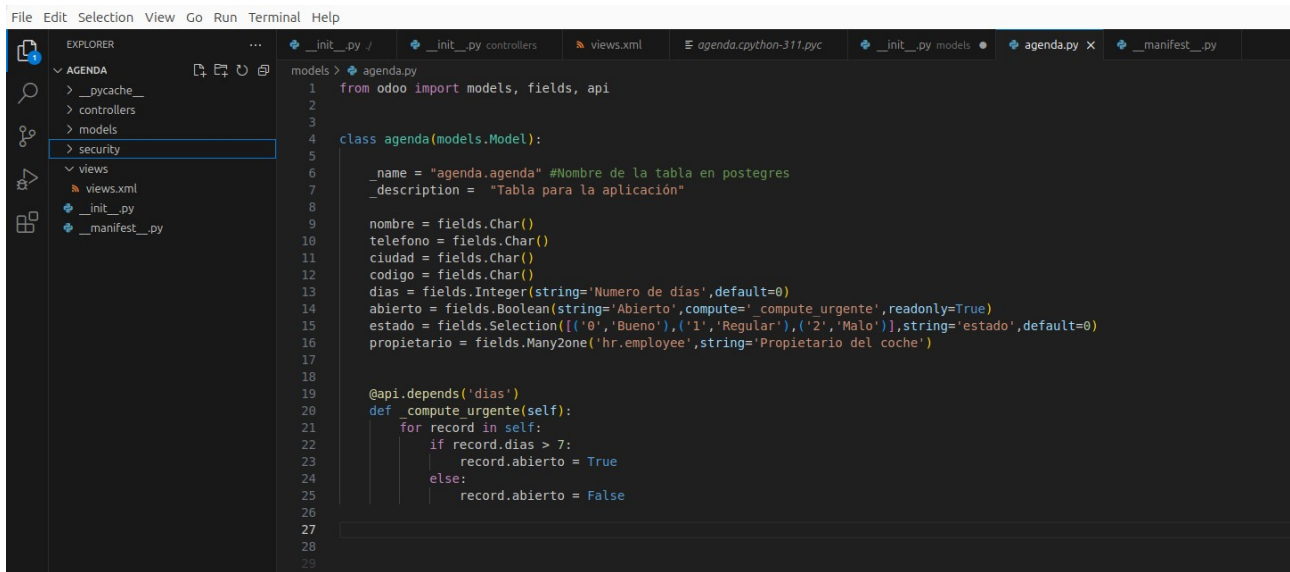


Figura 1: Entorno de desarrollo para trabajar con Python.

2. Estructura básica de un módulo

Cada aplicación y cada módulo de Odoo se almacena en una carpeta dentro del directorio addons. El nombre técnico de la aplicación o módulos se corresponden con el de la carpeta que lo alberga en el directorio addons (o en cualquier otro que se haya creado específicamente para guardar de forma separada módulos propios y que se haya declarado añadiendo su ruta absoluta en el fichero de configuración de Odoo).

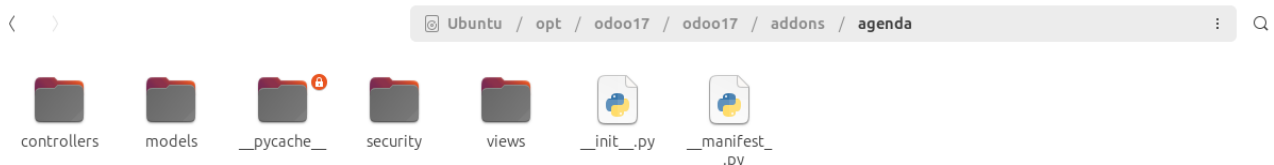


Figura 2: Estructura de un módulo.

En líneas generales un módulo de Odoo se compone de, al menos, dos archivos principales y una serie mínima de carpetas que obedecen a una estructura común:

Ficheros	__init__.py	Archivo principal del módulo y de carga del código Python
	__manifes__.py	Archivos de manifiesto del módulo con metadatos y declaraciones de ficheros XML que deben ser cargados.
Directorios básicos	models/	Contiene archivos Python (.py) para definición de las entidades (modelos, clases y métodos). Se aconseja un fichero por cada modelo.
	views/	Contiene archivos .xml y csv donde se especifican permisos de grupos y reglas de seguridad (listas de control de acceso). Suelen contener los archivos ir.model.access.csv y security.xml
	report/	Guarda modelos de informes (.xml) y archivos .py y .xml relacionados.
	controllers/	Contiene archivos de código para controladores y rutas http del sitio web.
	demo	Se usa para proporcionar datos de prueba precargados en ficheros .xml
	static	Para documentación para el sitio web en directorios img/css/, js/, lib/, etc. Este directorio contiene principalmente archivos JavaScript, hojas de estilo e imágenes.
Otros directorios	i18n/	Contiene archivos de localización y traducción.
	data/	Archivos css y xml que cargan datos al sistema (si tiene contenido se carga obligatoriamente).
	wizard/	Contiene vistas de tipo wizard. Reagrupa modelos transitorios.
	test/	Para incluir el código de test para pruebas de funcionamiento.

```
$ tree my_module
my_module/
├── __init__.py
├── __manifest__.py
├── controllers
│   ├── __init__.py
│   └── controllers.py
├── demo
│   └── demo.xml
├── models
│   ├── __init__.py
│   └── models.py
├── security
│   └── ir.model.access.csv
└── views
    ├── templates.xml
    └── views.xml
5 directories, 10 files
```

Figura 3: Estructura de las carpetas del módulo.

3. Archivo `__init__.py`

Archivo principal del módulo con instrucciones de importación; es el primer archivo que se lee en la ejecución e indica qué otros archivos Python se deben importar y ejecutar pues contiene las referencias a otros archivos .py a los directorios donde se encuentran. Cualquier directorio que contenga archivos Python debe incluir un archivo de este tipo, aunque esté vacío.

```
__init__.py
1  from . import models
2  from . import controllers
3
```

Figura 4: Fichero `__init__.py`

4. Archivo `__manifest__.py`

Se trata de otro archivo que contiene los datos básicos del módulo, como la descripción, información acerca del autor, las dependencias, así como los datos relacionados con la instalación, seguridad y la existencia o no de datos demo. Es un diccionario Python clave-valor con metadatos del modelo.

El orden en que se declaran los archivos .xml es importante. Si existieran dos archivos .xml uno para las vistas y otro para menús y acciones de ventana, sería necesario que primero se declarase el archivo de vistas y luego el archivo donde están las acciones, pues que ellas hacen referencia al id de las vistas.

```

1  {
2      "name": "Agenda", # Nombre del Modulo
3      "summary": "Herramienta Agenda", # Module subtitle phrase
4      "description": "Herramienta para manejar los teléfonos", # Supports reStructuredText(RST) format (description is Deprecated)
5      "version": "17.0.1.0.0",
6      "author": "Juan Maria Sanchez Curto",
7      "category": "Tools",
8      "license": "AGPL-3",
9      "depends": ["base"], #Modulos con los que voy a interactuar
10     "data": [
11         "security/ir.model.access.csv",
12         "#views/templates.xml",
13         "views/views.xml",
14     ],
15     # This demo data files will be loaded if db initialize with demo data (commented because file is not added in this example)
16     # 'demo': [
17         #     'demo.xml'
18     # ],
19
20 }

```

Figura 5: Fichero `__manifest__.py`

Elementos más importantes del fichero `__manifest__.py`

name: nombre del módulo
version: versión del módulo
description: una descripción del módulo.
author: persona o entidad que ha desarrollado el módulo.
Website: sitio web del módulo
license: tipo de licencia de módulo.
depends. Lista de módulos de los cuales depende el módulo. El módulo base es el más utilizado, puesto que en él se definen algunos datos que son necesarios par las vista
data: lista los fichero XML que se cargarán con la instalación del módulo.
installable. Determina si el módulo es intalable o no.

La nomenclatura seguida de Odoo está basada den un nombre jerárquico separado en ramas por cada modelo.

En un módulo llamado **mimodulo** sus dos modelos principales, **modelo1** y **modelo2**, se crearán en los ficheros **mimodulo_modelo1.py** y **mimodulo_modelo2.py**. A las vistas, menús y actions se las suele denominar añadiendo view con su tipo, action o menú como sufijo. Así se llamarán:

```

mimodulo.modelo1_view_form
mimodulo.modelo1_view_tree
mimodulo.modelo1_view_kanban

```

```

mimodulo.modelo1_action_window

```

```

mimodulo.menu_root
mimodulo.modelo1_menu con padre mimodulo.menu_root

```

5. Creación de un módulo paso a paso

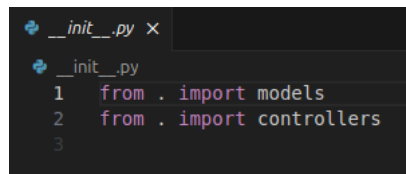
Se pretende crear un módulo llamado `hostel` para gestionar la información básica de un un hostel. Los pasos a seguir para realizar la creación del mismo son los siguientes:

Para cada módulo hay una carpeta con el nombre del módulo. Para crear un módulo tendremos que crear una carpeta dentro de la carpeta `addons` o bien crear una carpeta nueva para nuestros módulos y añadirla a la ruta `addons_path` siguiendo los pasos:

Paso 1. Crear el archivo de `__init__.py` y `__manifest__.py`

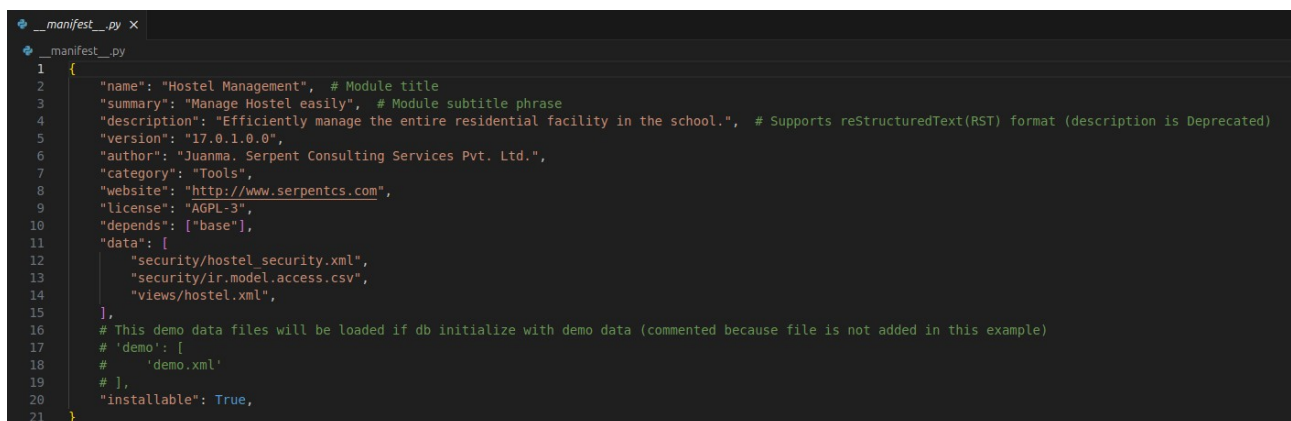
`__init__.py` . Este archivo le dice a Python que otros ficheros puede abrir.

`__manifest__.py`. Este el archivo tiene la descripción del módulo. Se trata de un fichero `.xml` con los pares clave-valor.



```
__init__.py
1 from . import models
2 from . import controllers
3
```

Figura 6: Fichero `__init__.py` del modelo.



```
__manifest__.py
1 {
2     "name": "Hostel Management", # Module title
3     "summary": "Manage Hostel easily", # Module subtitle phrase
4     "description": "Efficiently manage the entire residential facility in the school.", # Supports reStructuredText(RST) format (description is Deprecated)
5     "version": "17.0.1.0.0",
6     "author": "Juanma, Serpent Consulting Services Pvt. Ltd.",
7     "category": "Tools",
8     "website": "http://www.serpentcs.com",
9     "license": "AGPL-3",
10    "depends": ["base"],
11    "data": [
12        "security/hostel_security.xml",
13        "security/ir.model.access.csv",
14        "views/hostel.xml",
15    ],
16    # This demo data files will be loaded if db initialize with demo data (commented because file is not added in this example)
17    # 'demo': [
18    #     'demo.xml'
19    # ],
20    "installable": True,
21 }
```

Figura 7: Fichero `__manifest__.py` del modelo.

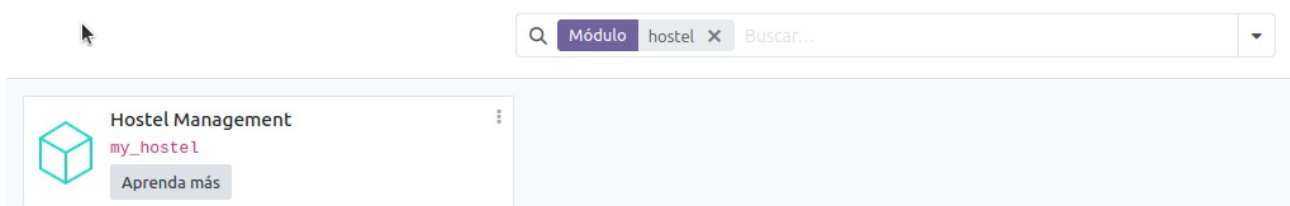


Figura 8: Modelo activo.

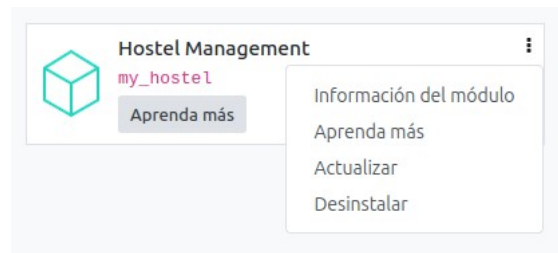


Figura 9: Información del módulo.

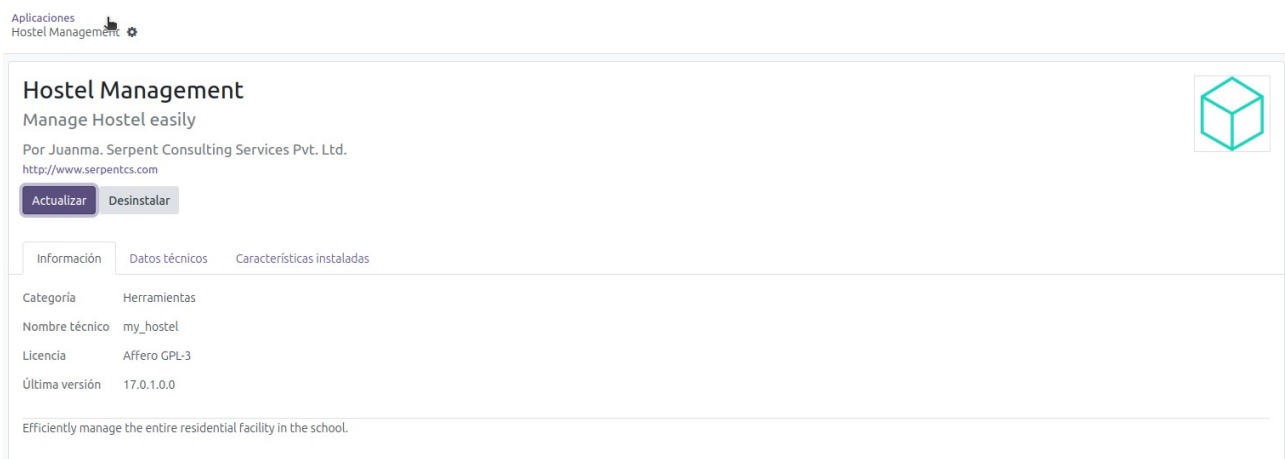


Figura 10: Módulo ya instalado.

Paso 2. Creación del resto de carpetas

Es de especial interés la carpeta **models**. En ella existe también un fichero **__init__.py**. En el caso de un modelo llamado **hostel**, el contenido del fichero será:

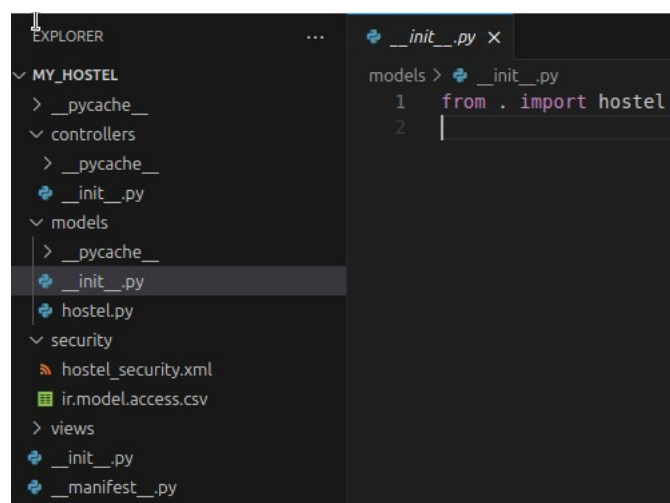
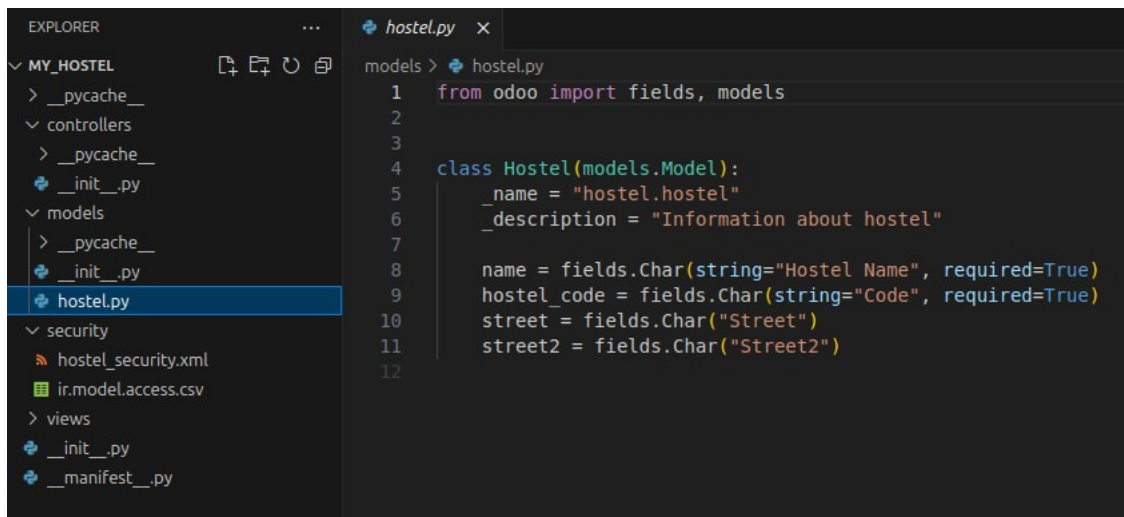


Figura 11: Configuración del proyecto.

Además existe también un fichero Python donde se define la clase correspondiente, en este caso será **hostel.py**.

ORM permite definir las tablas y las relaciones (1-1,1-N,N-M) por medio de objetos. Los objetos se declaran como clases Python extendiendo la clase Model.

Estos objetos del modelo pueden configurarse estableciendo una serie de atributos en su definición. El atributo más importante es **_name** que es obligatorio y define el nombre del modelo en el sistema de Odoo.



```
1 from odoo import fields, models
2
3
4 class Hostel(models.Model):
5     _name = "hostel.hostel"
6     _description = "Information about hostel"
7
8     name = fields.Char(string="Hostel Name", required=True)
9     hostel_code = fields.Char(string="Code", required=True)
10    street = fields.Char("Street")
11    street2 = fields.Char("Street2")
12
```

Figura 12: Modelo en Python

Los campos del modelo definen lo que el modelo puede almacenar y donde. Estos campos se definen como atributos de la clase del modelo:

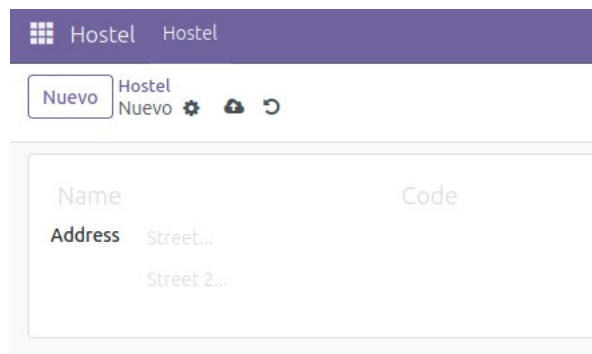
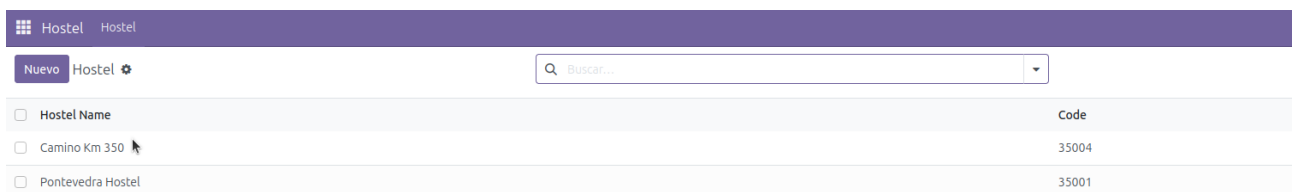


Figura 13: Inserción de un registro.



Hostel Name	Code
Camino Km 350	35004
Pontevedra Hostel	35001

Figura 14: Listado de registros.

Paso 3. Confección de Vistas

El siguiente paso para la creación de un módulo será la confección de Vistas asociadas a los modelos.

Hay que relacionar los modelos con las vistas y añadir los elementos de menú y las acciones de ventana. Es en este fichero (y otros ficheros como este que se pueden añadir) donde se relacionan.

Toda vista es un objeto de tipo record basado en `ir.ui.view` que se define unívocamente mediante un id que incluye el nombre del módulo, el del modelo y su tipo. Por ejemplo, `lapelicula.pelicula_tree` será una vista de tipo árbol (también llamado lista) del modelo película del módulo `lapelicula`.

```
views > hostel.xml
1 <odoo>
2
3 <!-- Tree(list) View -->
4 <record id="view_hostel_tree_view" model="ir.ui.view">
5   <field name="name">hostel.hostel.tree.view</field>
6   <field name="model">hostel.hostel</field>
7   <field name="arch" type="xml">
8     <tree>
9       <field name="name"/>
10      <field name="hostel_code"/>
11    </tree>
12  </field>
13 </record>
14
15 <!-- Form View -->
16 <record id="view_hostel_form_view" model="ir.ui.view">
17   <field name="name">hostel.hostel.form.view</field>
18   <field name="model">hostel.hostel</field>
19   <field name="arch" type="xml">
20     <form string="Hostel">
21       <sheet>
22         <div class="oe_title">
23           <h3>
24             <table>
25               <tr>
26                 <td style="padding-right:10px;"><field name="name" required="1"
27                   placeholder="Name" /></td>
28                 <td style="padding-right:10px;"><field name="hostel_code"
29                   placeholder="Code" /></td>
30               </tr>
31             </table>
32           </h3>
33         </div>
34         <group>
35           <group>
36             <label for="street" string="Address"/>
37             <div class="o_address_format">
38               <field name="street" placeholder="Street..." class="o_address_street"/>
39               <field name="street2" placeholder="Street 2..." class="o_address_street"/>
40             </div>
41           </group>
42         </group>
43       </sheet>
44     </form>
45   </field>
46 </record>
```

Figura 15: Vista del modelo.

```

47
48 <!-- Search View -->
49 <record id="view_hostel_search_view" model="ir.ui.view">
50   <field name="name">Hostel Search</field>
51   <field name="model">hostel.hostel</field>
52   <field name="arch" type="xml">
53     <search>
54       <field name="name"/>
55       <field name="hostel_code"/>
56     </search>
57   </field>
58 </record>
59
60 <record id="action_hostel" model="ir.actions.act_window">
61   <field name="name">Hostel</field>
62   <field name="type">ir.actions.act_window</field>
63   <field name="res_model">hostel.hostel</field>
64   <field name="view_mode">tree,form</field>
65   <field name="help" type="html">
66     <p class="oe_view_nocontent_create">
67       Create Hostel.
68     </p>
69   </field>
70 </record>
71
72 <menuitem id="hostel_main_menu" name="Hostel" sequence="1"/>
73
74 <menuitem id="hostel_type_menu" name="Hostel" parent="hostel_main_menu"
75   action="my_hostel.action_hostel" groups="my_hostel.group_hostel_manager"
76   sequence="1"/>
77
78 </odoo>

```

Figura 16: Vista del modelo.

Seguridad del módulo

Cuando se añade un nuevo modelo, se tiene que definir quien puede crear, actualizar, leer... registros. Esto puede suponer definir nuevos grupos. Si un usuario no tiene derechos de acceso, Odoo no le mostrará en ese caso menús ni vistas.

La seguridad que queremos añadir en este caso son:

- Todos pueden leer los registros del hostel.
- Un grupo nuevo de usuario llamado Hostel Manager tendrá derecho a crear, leer, actualizar y borrar registros.

Para implementar esto, se necesitan llevar a cabo los siguientes pasos:

1. Crear un fichero llamado security/hostel_security.xml con el siguiente contenido

```

<?xml version="1.0" encoding="utf-8"?>
<odoo>
<record id="module_category_hostel" model="ir.module.category">
    <field name="name">Hostel Management</field>
    <field name="sequence">31</field>
</record>

<record id="group_hostel_manager" model="res.groups">
    <field name="name">Hostel Manager</field>
    <field name="category_id" ref="module_category_hostel"/>
    <field name="users" eval="[(4, ref('base.user_root')), (4,
ref('base.user_admin'))]"/>
</record>

<record id="group_hostel_user" model="res.groups">
    <field name="name">Hostel User</field>
    <field name="category_id" ref="module_category_hostel"/>
</record>

</odoo>

```

Figura 17: hostel_security.xml

El fichero security/hostel_security.xml define un nuevo grupo de seguridad a partir de la creación de un registro en res.groups. También se dan derechos al usuario admin usando su id, base.user_admin para que el usuario admin tenga derechos en el módulo hostel.hostel.

2. Añadir un fichero llamado security/ir.model.access.csv con el contenido siguiente:

```

id,name,model_id:id,group_id:id,perm_read,perm_write,perm_
create,perm_unlink
access_hostel_manager_id,access.hostel.manager,my_hostel.model_
hostel_hostel,my_hostel.group_hostel_manager,1,1,1,1
access_hostel_user_id,access.hostel.user,my_hostel.model_hostel_
hostel,my_hostel.group_hostel_user,1,0,0,0

```

Figura 18: ir.model.access.csv

El fichero ir.model.access.csv asocia permisos a los grupos sobre modelos.

3. Añadir las líneas al fichero __manifest__.py

```
# ...  
"data": [  
    "security/hostel_security.xml",  
    "security/ir.model.access.csv",  
    "views/hostel.xml",  
],  
# ...
```

Figura 19: Líneas nueva del modelo.

Es importante el orden de estas líneas en el fichero de manifiesto. Esto se debe a porque los grupos de seguridad tienen que ser cargados antes de que se les puedan asignar permisos.